

2. Agentic AI Application Architecture

이 워크숍에서는 에이전트 기반 AI 애플리케이션을 사용하여 여행을 예약합니다. 이 섹션에서는 애플리케이션 아키텍처를 살펴보고 애플리케이션에서 사용하는 주요 LangChain 및 LangGraph 개념을 중점적으로 다룹니다.

[!NOTE] LangChain과 LangGraph 비교

LangChain은 프롬프트, 도구, 모델 통합 등 대규모 언어 모델 작업을 위한 핵심 구성 요소를 제공합니다.

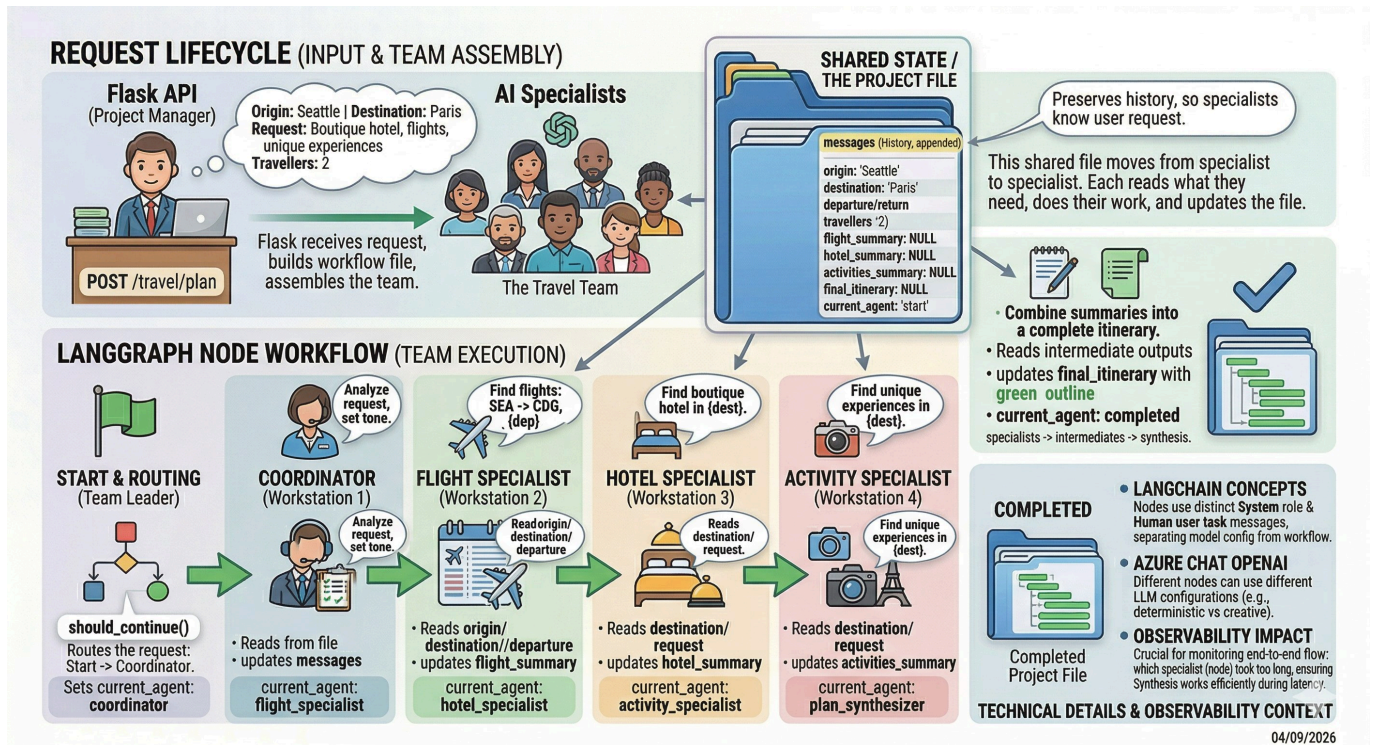
LangGraph는 이러한 개념을 기반으로 구성 요소 간의 복잡하고 상태 저장적인 워크플로를 조율합니다.

간단히 말해, LangChain은 LLM 기반 단계의 기능을 정의하는 데 도움을 주고, LangGraph는 에이전트 기반 애플리케이션에서 이러한 단계들이 어떻게 함께 흐르는지를 제어하는 데 도움을 줍니다.

이번 워크숍의 주된 목표는 OpenTelemetry를 사용하여 애플리케이션에 계측 도구를 구축하는 것이지만, 애플리케이션 구조에 대한 기본적인 이해는 관찰 가능성 작업을 훨씬 더 명확하게 만들어 줄 것입니다. 에이전트, 도구 및 워크플로가 어떻게 구축되었는지 살펴보면 시스템 추적 및 분석을 시작했을 때 텔레메트리가 무엇을 나타내는지 파악하는 데 도움이 될 것입니다.

아키텍처를 살펴보는 동안 구현 방식도 함께 살펴보고 싶으시다면, 애플리케이션 소스 코드는 EC2 인스턴스의 다음 경로에서 확인하실 수 있습니다.

~/workshop/agentic-ai/base-app/main.py.



애플리케이션 아키텍처 이 애플리케이션은 Flask API 로, 여행 계획 요청을 받아 여러 개의 LangChain 기반 LLM 노드로 구성된 LangGraph 워크플로우를 통해 실행합니다. 각 노드는 특정 역할을 수행하고, 공유 상태를 업데이트하며, 다음 단계로 넘어갑니다.

워크숍의 이 부분에서는 다음 내용을 검토하겠습니다.

- 요청 수명 주기
- 공유 상태 모델
- LangGraph 노드의 작동 방식

- 코드에서 사용된 LangChain 추상화
- 나중에 관측 가능성이 중요해질 것입니다.

2-1. 요청 수명 주기 (Request Lifecycle)

간략히 설명하면, 이 애플리케이션은 요청을 받아들여 여러 단계로 이루어진 워크플로로 변환합니다.

조정자 항공 전문가 호텔 전문가 활동 전문가 신디사이저

주요 흐름은 다음과 같습니다.

```
@app.route("/travel/plan", methods=["POST"])
def plan():
    data = request.get_json()

    origin = data.get("origin", "Seattle")
    destination = data.get("destination", "Paris")
    user_request = data.get(
        "user_request",
        f"Planning a week-long trip from {origin} to {destination}. "
        "Looking for boutique hotel, flights and unique experiences.",
    )
    travellers = int(data.get("travellers", 2))

    result = plan_travel_internal(
        origin=origin,
        destination=destination,
        user_request=user_request,
        travellers=travellers
    )

    return jsonify(result), 200
```

1. Flask는 요청을 수신합니다.
2. plan_travel_internal()워크플로 상태를 구축합니다.
3. LangGraph는 노드를 실행합니다.
4. 각 노드는 상태를 업데이트합니다.
5. 최종 여정표는 JSON 형식으로 반환됩니다.

2-2. 공유 상태 (Shared State)

이 앱에서 가장 중요한 LangGraph 개념은 공유 상태 객체입니다.

```
class PlannerState(TypedDict):
    messages: Annotated[List[AnyMessage], add_messages]
    user_request: str
    session_id: str
    origin: str
```

```

destination: str
departure: str
return_date: str
travellers: int
flight_summary: Optional[str]
hotel_summary: Optional[str]
activities_summary: Optional[str]
final_itinerary: Optional[str]
current_agent: str

```

이 상태는 그래프를 통해 노드에서 노드로 이동합니다.

각 노드:

- 상태에서 값을 읽습니다.
- 일을 줌 합니다
- 새로운 값을 상태에 다시 기록합니다.
- `current_agent`를 설정하여 다음에 발생할 일을 제어합니다.

이것은 LangGraph의 핵심적인 사고 모델입니다: 상태 기반 워크플로 오케스트레이션

2-3. 오케스트레이션

주요 오케스트레이션은 이 부분에서 이루어집니다: `plan_travel_internal()`

```

def plan_travel_internal(
    origin: str,
    destination: str,
    user_request: str,
    travellers: int,
) -> Dict[str, object]:
    session_id = str(uuid4())
    departure, return_date = _compute_dates()

    initial_state: PlannerState = {
        "messages": [HumanMessage(content=user_request)],
        "user_request": user_request,
        "session_id": session_id,
        "origin": origin,
        "destination": destination,
        "departure": departure,
        "return_date": return_date,
        "travellers": travellers,
        "flight_summary": None,
        "hotel_summary": None,
        "activities_summary": None,
        "final_itinerary": None,
        "current_agent": "start",
    }

```

```

workflow = build_workflow()
compiled_app = workflow.compile()

for step in compiled_app.stream(initial_state, config):
    node_name, node_state = next(iter(step.items()))
    final_state = node_state

```

이 함수는 다음과 같은 애플리케이션 수명 주기를 구현합니다.

- 초기 상태 구축
- 그래프를 생성하세요
- 컴파일하세요
- 스트림 실행 단계별 설명

2-4. 그래프 정의 (Defining the Graph)

그래프는 `build_workflow()` 에서 명시적으로 정의됩니다

```

def build_workflow() -> StateGraph:
    graph = StateGraph(PlannerState)
    graph.add_node("coordinator", lambda state: coordinator_node(state))
    graph.add_node("flight_specialist", lambda state:
flight_specialist_node(state))
    graph.add_node("hotel_specialist", lambda state:
hotel_specialist_node(state))
    graph.add_node("activity_specialist", lambda state:
activity_specialist_node(state))
    graph.add_node("plan_synthesizer", lambda state:
plan_synthesizer_node(state))
    graph.add_conditional_edges(START, should_continue)
    graph.add_conditional_edges("coordinator", should_continue)
    graph.add_conditional_edges("flight_specialist", should_continue)
    graph.add_conditional_edges("hotel_specialist", should_continue)
    graph.add_conditional_edges("activity_specialist", should_continue)
    graph.add_conditional_edges("plan_synthesizer", should_continue)
    return graph

```

아래는 라우팅 로직입니다

```

def should_continue(state: PlannerState) -> str:
    mapping = {
        "start": "coordinator",
        "flight_specialist": "flight_specialist",
        "hotel_specialist": "hotel_specialist",
        "activity_specialist": "activity_specialist",
        "plan_synthesizer": "plan_synthesizer",

```

```
}
return mapping.get(state["current_agent"], END)
```

- start
- coordinator
- flight specialist
- hotel specialist
- activity specialist
- synthesizer
- end

2-5. 노드 정의

이 앱에서 LangGraph 노드는 상태를 입력받아 업데이트된 상태를 반환하는 파이썬 함수일 뿐입니다.

예를 들어, 항공 전문가를 생각해 보세요.

```
def flight_specialist_node(state: PlannerState) -> PlannerState:
    llm = _create_llm(
        "flight_specialist", temperature=0.4, session_id=state["session_id"]
    )

    step = (
        f"Find an appealing flight from {state['origin']} to "
        f"{state['destination']} "
        f"departing {state['departure']} for {state['travellers']} "
        f"travellers."
    )

    messages = [
        SystemMessage(content="You are a flight booking specialist. Provide "
        "concise options."),
        HumanMessage(content=step),
    ]

    result = llm.invoke(messages)
    state["flight_summary"] = result.content
    state["messages"].append(result)
    state["current_agent"] = "hotel_specialist"
    return state
```

이는 일반적인 노드 패턴을 보여줍니다.

1. LLM을 생성하거나 액세스합니다.
2. 구조화된 상태에서 프롬프트를 생성합니다.
3. 모델을 호출합니다
4. 결과를 상태에 저장합니다.

5. 다음 노드를 설정합니다

호텔 및 활동 노드는 동일한 구조를 따르므로 워크플로를 쉽게 설명할 수 있습니다.

2-6. 메시지 추상화 (Message Abstractions)

이 애플리케이션은 하나의 긴 프롬프트 문자열 대신 LangChain 메시지 추상화를 사용합니다.

```
from langchain_core.messages import (
    AIMessage,
    BaseMessage,
    HumanMessage,
    SystemMessage,
)
```

이는 각 노드가 분리될 수 있기 때문에 중요합니다.

- 시스템 역할
- 사용자 작업
- 모델 응답

예를 들어:

```
messages = [
    SystemMessage(content="You are a flight booking specialist. Provide concise options."),
    HumanMessage(content=step),
]
result = llm.invoke(messages)
```

2-7. LLM 생성

LLM 생성은 이부분에서 만들어집니다.

```
def _create_llm(agent_name: str, *, temperature: float, session_id: str) -> ChatOpenAI:
    """Create an ChatOpenAI instance."""

    model_name = os.getenv("OPENAI_MODEL_NAME", "gpt-4.1-mini")

    return ChatOpenAI(
        model = model_name,
        temperature = temperature,
```

```

        # Uses OPENAI_API_KEY automatically from environment
    )

```

이 접근 방식은 모델 구성과 워크플로 로직을 분리합니다. 각 노드는 얼마나 결정론적이어야 하는지 또는 얼마나 창의적이어야 하는지에 따라 서로 다른 온도를 사용할 수 있습니다.

2-8. 분해 패턴 (Decomposition Pattern)

Synthesizer 단계에서 전문가들의 의견을 종합하여 하나의 답을 도출합니다.

```

def plan_synthesizer_node(state: PlannerState) -> PlannerState:
    llm = _create_llm(
        "plan_synthesizer", temperature=0.3, session_id=state["session_id"]
    )

    content = json.dumps(
        {
            "flight": state["flight_summary"],
            "hotel": state["hotel_summary"],
            "activities": state["activities_summary"],
        },
        indent=2,
    )

    response = llm.invoke(
        [
            SystemMessage(
                content="You are the travel plan synthesiser. Combine the
specialist insights into a concise, structured itinerary."
            ),
            HumanMessage(
                content=(
                    f"Traveller request: {state['user_request']}\n\n"
                    f"Origin: {state['origin']} | Destination:
{state['destination']}\n"
                    f"Dates: {state['departure']} to
{state['return_date']}\n\n"
                    f"Specialist summaries:\n{content}"
                )
            ),
        ]
    )
    state["final_itinerary"] = response.content
    state["messages"].append(response)
    state["current_agent"] = "completed"
    return state

```

이런 흐름은 에이전트형 앱에서 흔히 볼 수 있는 패턴입니다.

- 업무를 전문가별로 세분화하다
- 중간 출력물을 수집합니다
- 최종 답변으로 종합하다

이것이 바로 이 개요에서 여러분이 얻어야 할 구조 내용중 하나입니다.

Module 3. Agentic AI Application Architecture DONE!